

Furnace State Collector

Revised Requirements Document — Debug & Test Mode

1. Purpose and Scope

The Furnace State Collector continuously monitors the logical state of a furnace via GPIO inputs on a Raspberry Pi Zero 2 W.

It:

- Reads discrete GPIO inputs representing furnace states
 - Filters and debounces signals to remove glitches
 - Combines inputs into a single logical 10-bit state word
 - Detects stable state changes
 - Assigns a unique, monotonic epoch to each transition
 - Logs transitions to SQLite
 - Publishes transitions via MQTT
 - Provides metadata to decode individual bits for dashboards (e.g., Grafana)
 - Supports DEBUG diagnostics for visibility
 - Supports TEST mode for operator-driven simulated input
-

2. System Environment

2.1 Hardware

- Platform: Raspberry Pi Zero 2 W
- Inputs: Up to 10 discrete GPIOs (BCM numbering)
- Inputs are slow-changing (~1000 transitions/day)
- Input rise/fall times may be hundreds of ms

2.2 Software

- OS: Raspberry Pi OS (Lite)
- Python 3.x with standard libraries

- Additional packages: `RPi.GPIO`, `paho-mqtt`
-

3. Furnace State Model

3.1 State Word

- **10-bit unsigned integer**
- Each bit corresponds to a physical GPIO input
- 0 = inactive, 1 = active

3.2 Bit-to-GPIO Mapping

Bit GPIO

furn0 5

furn1 6

furn2 12

furn3 13

furn4 16

furn5 19

furn6 20

furn7 21

furn8 26

furn9 4

3.3 Bit Masking

- Bit mask defines which bits are **active/used**
- Masked-out bits:
 - Are ignored for state change detection
 - Do not trigger logging or MQTT publishing
- Effective state:

```
effective_state = raw_state & used_mask
```

4. Bit Definitions

4.1 Configuration File (`state.cfg`)

- CSV-style key/value lines:

```
bit=0,name=blower,used=1,desc=Circulation blower
bit=1,name=igniter,used=1,desc=Igniter active
...
```

- Fields:
 - `bit` (0–9)
 - `name` (short label)
 - `used` (0/1)
 - `desc` (description)

4.2 Metadata Reconciliation

- On startup:
 - Load `state.cfg`
 - Create `furnace_bits` table if missing
 - **Overwrite database entries if definitions differ** (correct typos/rename bits)
-

5. Sampling and Debouncing

- Synchronous sampling of all bits
 - Configurable sampling interval (default 50 ms)
 - Debounce: a candidate state must persist **N samples** (default 5) before acceptance
 - Debounce applied to the **entire state word**
 - Ensures immunity to slow rise/fall times or brief glitches
-

6. State Change Detection

- Tracks:
 - `stable_state` (current accepted state)
 - `candidate_state` (potential new state)
 - `candidate_count` (stable sample count)
 - A state change is recorded **only when candidate_count ≥ DEBOUNCE_COUNT**
 - Initial state at startup is **always logged**
-

7. Timestamping and Epoch

- Epoch = integer seconds since Unix epoch
 - Monotonic and unique:
 - If current wall clock $\leq$ last epoch $\rightarrow$ increment last epoch by 1
 - Guarantees uniqueness for DB primary key and MQTT messages
-

8. Database Requirements (SQLite)

8.1 Tables

furnace_state

```
epoch INTEGER PRIMARY KEY  
state INTEGER NOT NULL
```

furnace_bits

```
bit INTEGER PRIMARY KEY  
name TEXT NOT NULL  
description TEXT  
used INTEGER NOT NULL  
gpio INTEGER
```

8.2 Views

- `furnace_state_bits`:
 - Expands `state` word into per-bit values for dashboard access
 - Filters to used bits only

8.3 Behavior

- Tables created **if missing**
 - Metadata reconciled with `state.cfg`
 - No manual setup required
-

9. MQTT Publishing

- Single topic (configurable)
- Payload: JSON with epoch and state

- Metadata may optionally be published at startup for decoding
-

10. Startup and Restart Behavior

1. Load configuration
 2. Initialize GPIO (unless TEST mode)
 3. Initialize/reconcile database
 4. Initialize MQTT
 5. Read current state (GPIO or TEST input)
 6. Log initial state
 7. Begin monitoring loop
- Handles power loss, restarts, and clock adjustments gracefully
-

11. Debug Mode

- `DEBUG = True` enables detailed diagnostic prints
 - Prints include:
 - Loaded bit definitions
 - Bit mask
 - GPIO mapping
 - Initial state
 - Candidate state detections and reaffirmations
 - Accepted state transitions
 - Database insertions
 - MQTT publishing
 - `DEBUG = False` suppresses diagnostics
-

12. Test Mode

- `TEST_MODE = True` overrides GPIO readings
- Operator enters **2-digit hex values** representing the full state word

- Keyboard input drives the collector exactly as if physical GPIOs changed
 - Same debounce, epoch, DB, and MQTT logic applied
 - Prints raw input and masked state for clarity
 - `TEST_MODE = False` → normal GPIO operation
-

13. Configuration / Tunables

- Top-of-file parameters:
 - `SAMPLE_INTERVAL`
 - `DEBOUNCE_COUNT`
 - `MQTT_BROKER`, `MQTT_PORT`, `MQTT_TOPIC`
 - `DB_PATH`
 - `DEBUG / TEST_MODE`
-

14. Error Handling

- Invalid TEST input: prompt user again
 - GPIO read errors: ignored or retried
 - MQTT failures: logged but do not block DB
 - SQLite errors: logged and retried if possible
-

15. Non-Goals

- No sub-second timestamping
 - Does not control furnace
 - Does not log raw GPIO edges
 - Does not attempt complex analytics
-

16. Design Philosophy

- **State-based reasoning** over edge-based

- **Deterministic behavior**
- **Metadata-driven** for flexibility
- **Debug/Test modes** for safe operator testing and troubleshooting